

ЛЕКЦИЯ 3

Супервизорное управление: синтез логики по спецификации

Логику управления не пишут — её вычисляют из модели оборудования и списка запретов

90 минут · две части по 45

Курс «Управление робототехническими комплексами»

План занятия

- 1 Почему написанная вручную логика обречена
- 2 Управляемые и неуправляемые события
- 3 Типовые ошибки классификации
- 4 Супервизор, допустимость, управляемая система
- 5 Теорема Рамаджа–Вонэма о существовании

- 6 Супремальный управляемый подязык
- 7 Алгоритм синтеза и его сходимость
- 8 Обратное распространение по неуправляемым событиям
- 9 Модульный и децентрализованный синтез
- 10 Частичная наблюдаемость: где ломается единственность

Синие — первая часть · янтарные — вторая часть

ЧАСТЬ 1

Синтез вместо программирования

Инженер описывает, что оборудование может делать и что делать запрещено. Логику вычисляет алгоритм.

3.1 Два дефекта ручной логики

Корректность не проверяема

- Логика правильна настолько, насколько автор учёл сочетания
- Для $5 \cdot 10^4$ состояний это невозможно
- Дефекты проявляются как редкие «сбои»
- Воспроизводимого сценария нет

Изменения разрушают корректность

- Новое ограничение взаимодействует со всеми прежними
- Добавление робота требует пересмотра всей логики
- После 3–4 итераций «латания» логика неподдерживаема
- Каждая правка порождает новый тупик в другом месте

“

Среди всех корректных супервизоров существует единственный наименее ограничительный, и он вычислим.

Главный результат теории Рамаджа–Вонэма

3.2

Управляемые и неуправляемые события

- Σ_c — можно предотвратить: команды пуска, движения, захвата
- Σ_{uc} — предотвратить невозможно: завершения, отказы, поступления, действия человека
- Супервизор может **не выдать команду**, но не может **отменить факт**
- Правило: управляемы начала действий
- Неуправляемы завершения, отказы и внешние воздействия

Ключевая идея

Этого различения нет в базовом курсе автоматов, и оно составляет основной вклад теории ДЭС в инженерную практику. Всё дальнейшее — следствие.



Классифицируем

К какому классу относится событие `rel1` — освобождение зоны манипулятором по завершении движения?

A

Управляемое: супервизор решает, когда отпустить зону

B

Неуправляемое: происходит по завершении движения

C

Зависит от реализации

D

Не является событием



Классифицируем

К какому классу относится событие rel_1 — освобождение зоны манипулятором по завершении движения?

A

Управляемое: супервизор решает, когда отпустить зону

B

Неуправляемое: происходит по завершении движения

C

Зависит от реализации

D

Не является событием

Ответ. Неуправляемое. Освобождение наступает по завершении движения; супервизор не может «удерживать» зону занятой — это не его решение. Классификация существенно влияет на результат синтеза.

3.3

Три типовые ошибки классификации

Ошибка	Почему возникает	Последствие
Отказ объявлен управляемым	«мы просто запретим отказ привода»	супервизор синтезируется, но неосуществим
Завершение операции управляемо	путают move_start и move_done	логика опирается на несуществующую возможность
Действия человека игнорируются	«оператор не будет так делать»	логика некорректна при реальной эксплуатации

Пустой результат синтеза чаще указывает на ошибку классификации, чем на противоречие спецификаций

3.4 Супервизор и допустимость

- $S : L(G) \rightarrow 2^\Sigma$ — множество разрешённых событий после трассы
- Допустимость: $\Sigma_{uc} \cap \Gamma_G(f(x_0, s)) \subseteq S(s)$
- Всё физически возможное и неуправляемое обязано быть разрешено
- $L(S/G)$ — язык системы под управлением
- Неблокирующий супервизор: $L_m(S/G) = L(S/G)$

Ключевая идея

Условие допустимости — формализация тезиса «супервизор не может отменить факт». Именно оно отличает задачу супервизорного синтеза от обычного ограничения языка.

$$K \cdot \Sigma_{uc} \cap L(G) \subseteq K$$

- Из любого достижимого состояния управляемой системы
- всякое физически возможное неуправляемое событие
- не выводит за пределы допустимого поведения
- Проверяется формально; при неудаче указывает на дефект проекта

Условие реализуемости спецификации

$\exists$ допустимый S с $L(S/G) = K \Leftrightarrow K$ управляем

- Необходимость: допустимость требует $s\sigma \in K$ для неуправляемых σ
- Достаточность: строим $S(s) = \{ \sigma : s\sigma \in K \}$ явно
- Точный критерий реализуемости спецификации
- Если K неуправляем — точная реализация невозможна

3.7

Что делать, если спецификация неуправляема

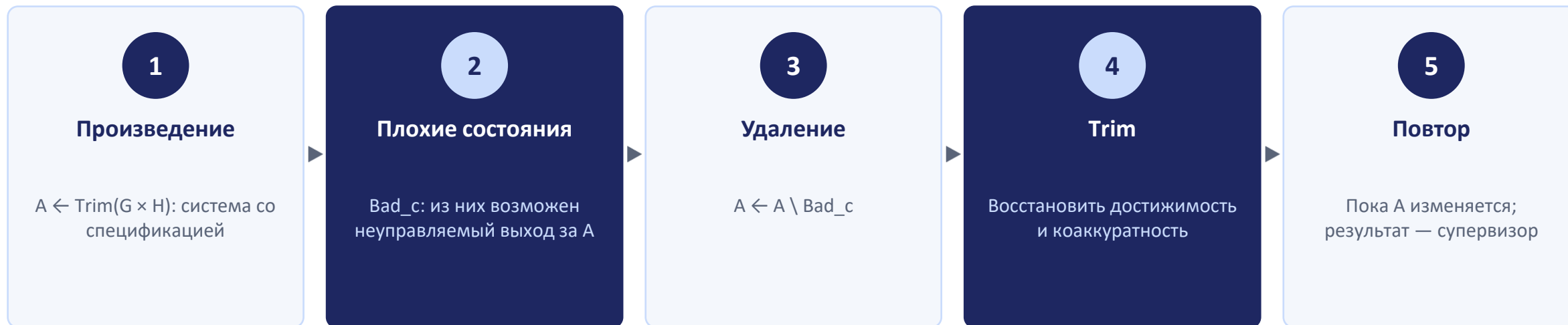
- Точная реализация невозможна — но это не тупик
- Ищем наибольший управляемый подязык $K \subseteq E$
- Класс $C(E)$ управляемых подязыков замкнут относительно объединения
- Значит, существует наибольший элемент $K \uparrow = \sup C(E)$
- **Супремальный управляемый подязык** — единственный наилучший ответ

Ключевая идея

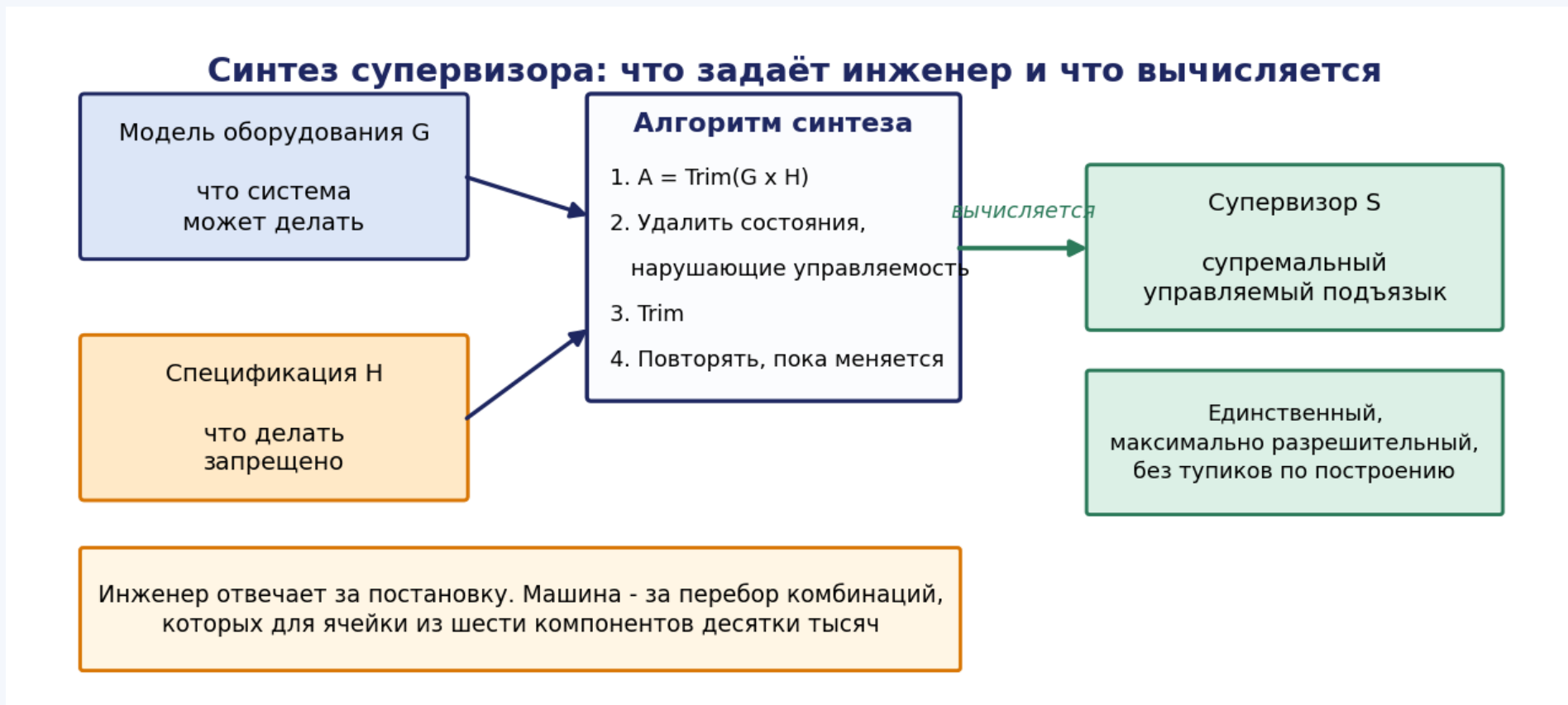
Для любой, даже нереализуемой, спецификации существует единственное наилучшее приближение снизу. Результат синтеза не зависит от искусства проектировщика.

3.8

Алгоритм синтеза



Сходится не более чем за $|X| \cdot |Y|$ итераций; сложность полиномиальная



Инженер задаёт модель оборудования и список запретов. Логика вычисляется.

3.9

Почему алгоритм корректен

- Сходимость: множество A строго убывает и конечно
- Монотонность: $K \uparrow$ содержится в A_k на каждом шаге
- Ни одно «нужное» состояние не удаляется — доказывается индукцией
- По завершении Bad_c пуст, значит $L(A)$ управляем
- A обрезан, значит неблокирующий; отсюда $L_m(A) = K \uparrow$

? Оценим

Синтез для ячейки из 6 компонентов выдал пустой супервизор. Какова наиболее вероятная причина?

- A** Спецификации противоречивы
- B** Ошибка в классификации событий
- C** Модель слишком велика для алгоритма
- D** Ячейка физически неработоспособна

? Оценим

Синтез для ячейки из 6 компонентов выдал пустой супервизор. Какова наиболее вероятная причина?

A Спецификации противоречивы

B Ошибка в классификации событий

C Модель слишком велика для алгоритма

D Ячейка физически неработоспособна

Ответ. На практике чаще всего вторая: событие ошибочно объявлено неуправляемым, и алгоритм не находит точки вмешательства. Проверять начинают с таблицы классификации.

П Е Р Е Р Ы В

Конец первой части

Мы получили аппарат синтеза. После перерыва — как он работает на живой ячейке и что делать с большими системами.

ЧАСТЬ 2

Синтез на живой ячейке

Что алгоритм находит такого, чего не находит человек, и как масштабировать метод.

3.10

Постановка задачи для разбора

- Два манипулятора M_i : $I_i \rightarrow \text{start}_i \rightarrow W_i \rightarrow \text{fin}_i \rightarrow D_i \rightarrow \text{out}_i \rightarrow I_i$
- $\text{start}_i \in \Sigma_{\text{с}}$ — команда пуска
- $\text{fin}_i \in \Sigma_{\text{ус}}$ — операция завершается сама
- $\text{out}_i \in \Sigma_{\text{ус}}$ — вывод инструмента механически необратим
- Система $G = M_1 \parallel M_2$, девять состояний

3.11 Наивное рассуждение и его провал

- Спецификация: запрещено состояние (D_1, D_2) — оба выводят инструмент
- Кажется: достаточно не пускать второго, пока первый в D
- Но (W_1, W_2) допустимо спецификацией
- Из него неуправляемое fin_1 ведёт в (D_1, W_2) — тоже допустимо
- Далее неуправляемое fin_2 ведёт в (D_1, D_2) — **запрещено, но неотвратимо**

Кейс

Оба события неуправляемы, предотвратить переход невозможно. Значит, спецификация неуправляема, и вмешиваться нужно было раньше — на этапе разрешения start_2 .

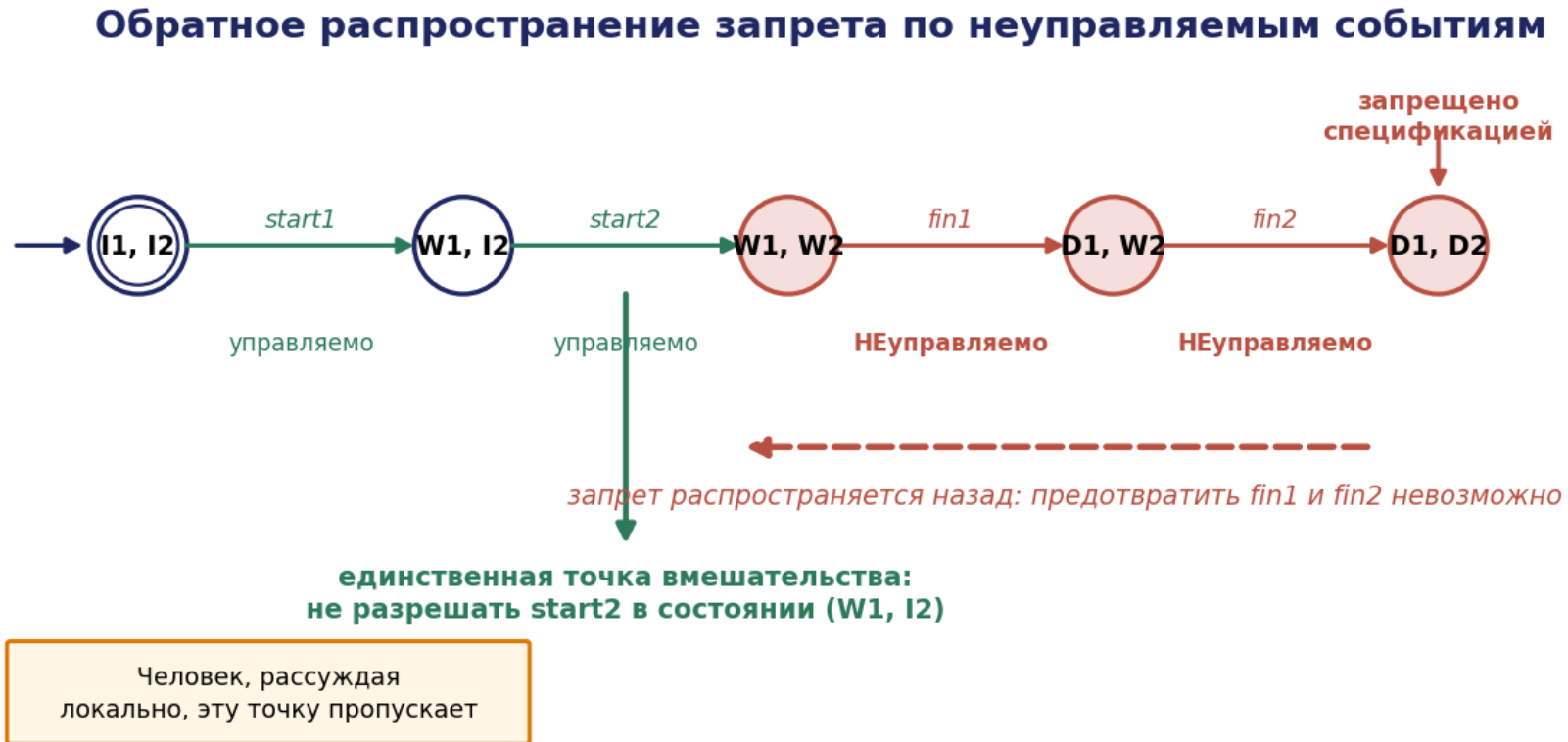
3.12 Обратное распространение по неуправляемым событиям

$$\text{Bad}^* = \{ q : \exists u \in \Sigma_{uc}^*, f(q, u) \in \text{Bad} \}$$

- Если состояние запрещено, запрещены и все, откуда оно достижимо неуправляемо
- Множество плохих состояний замкнуто относительно неуправляемого обратного достижения
- Именно это вычисляют шаги 4–5 алгоритма
- Повторение цикла — из взаимодействия с требованием неблокируемости

Алгоритм находит точку вмешательства, которую человек пропускает

3.12 Обратное распространение запрета



Единственная точка вмешательства находится за два шага до нарушения - её человек и пропускает.

3.13 Что даёт синтез: цифры

1

единственный супремальный
супервизор

$O(|X|^2|Y|^2|\Sigma|)$

сложность алгоритма

2–3

типичное число итераций
внешнего цикла

0

тупиков в результате по
построению

- Основное ограничение — размер $|X|$, экспоненциальный по числу компонентов
- Символьная реализация (Supremica) поднимает границу на порядки

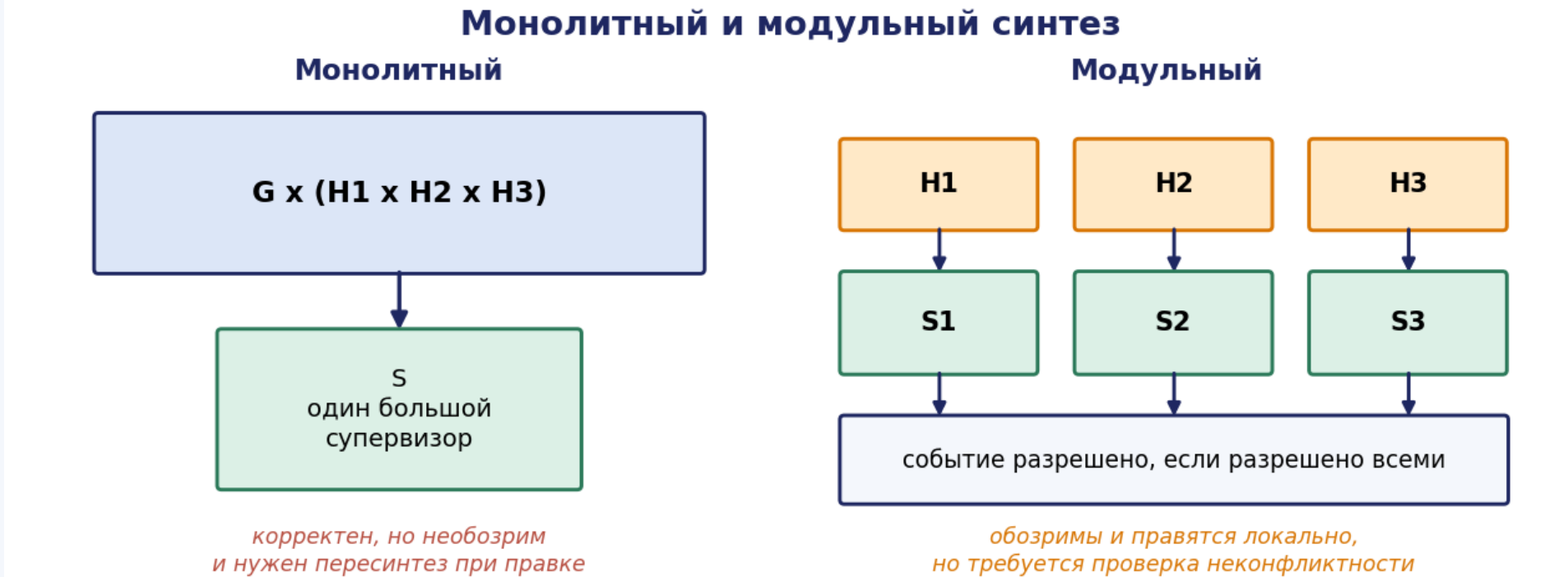
3.14 Модульный синтез

- Спецификация разбивается на простые ограничения $E_1, \dots, E_m$
- Для каждого — свой супервизор над малым фрагментом системы
- Событие разрешено, если разрешено всеми
- Каждый S_j обозрим и легко правится при изменении требования
- Но: совокупность неблокирующих супервизоров может блокировать

Ключевая идея

Условие неконфликтности: $(K_1 \cap \dots \cap K_m)^- = K_1 \cap \dots \cap K_m$. Нарушение означает трассу, после которой каждый супервизор «считает», что задачу можно завершить, а совместно завершить нельзя.

3.14 Монолитный и модульный синтез



Модульные супервизоры обозримы и правятся локально, но требуют проверки неконфликтности.

3.15 Шаблоны спецификаций

Шаблон	Формулировка	Размер автомата
Взаимное исключение	ресурс R — не более одного потребителя	$1 + n$ состояний
Ёмкость буфера	число деталей в $B \leq k$	$k+1$ состояний
Порядок операций	операция b только после a	2 состояния
Запрет состояния	комбинация (x,y) недостижима	произведение с удалением
Приоритет	при конфликте предпочитается i	зависит от постановки

Каждое ограничение — отдельный автомат над минимально необходимым подалфавитом

3.16 Децентрализованное управление

- Единый супервизор нереализуем для распределённого комплекса
- n локальных супервизоров, каждый со своим наблюдением и управлением
- Событие разрешено, если разрешено всеми, кто может его контролировать
- Условие реализуемости — **соуправляемость**
- Проверка дороже управляемости, но выполняема

Ключевая идея

Если спецификация не соуправляема, децентрализованное решение невозможно при данном распределении датчиков и исполнительных механизмов. Это формальный критерий для вопроса «сколько связи нужно между ячейками».

3.17 Частичная наблюдаемость: единственность теряется

- Наблюдаемость: решения одинаковы для неразличимых трасс
- Класс наблюдаемых подязыков **не замкнут** относительно объединения
- Супремального наблюдаемого подязыка в общем случае нет
- Максимальных вариантов несколько, и они несравнимы
- Проектировщик обязан сделать выбор

Факт

Это качественное усложнение: при полном наблюдении алгоритм даёт единственный ответ, при частичном — несколько несравнимых максимальных решений.

Нормальность как практическая замена

- Нормальность: $K = P^{-1}(P(K)) \cap L(G)$
- Принадлежность определяется исключительно проекцией трассы
- Нормальность влечёт наблюдаемость, но сильнее её
- Класс нормальных подязыков замкнут — супремум существует и вычислим
- Если $\Sigma_{uc} \subseteq \Sigma_o$, приближение близко к оптимальному

Ключевая идея

Условие «все неуправляемые события наблюдаемы» — хороший проектный ориентир при выборе датчиков. Отказы и завершения обычно и так снабжены датчиками, поэтому условие часто выполнимо.

3.19 Методика синтеза: восемь шагов



Реализация не должна содержать логики сверх таблиц супервизора

Проверка ограничительности

- Сравнить $L_m(S/G)$ с $L_m(G)$: какие полезные сценарии запрещены
- Чрезмерная ограничительность — признак проблемы, а не свойство метода
- Причина 1: излишне сильная спецификация
- Причина 2: консервативная классификация событий как неуправляемых
- Супремальность гарантирует: меньше запретить нельзя

3.21 Границы применимости метода

Ограничение	Суть	Обход
Размер модели	явное произведение до 10^6 – 10^7	символьные методы, модульность
Отсутствие времени	отвечает «возможно ли», не «успеет ли»	проверка дедлайнов отдельно, гл. 7 и 15
Отсутствие данных	события, а не значения	дискретизация или расширенные автоматы
Корректность модели	гарантии относятся к модели	валидация модели, гл. 17

Честное указание границ — часть инженерной культуры применения метода



Разберём случай

Спецификация «дверь не открывается во время работы станка». `open_door` — действие оператора. Управляема ли спецификация?

A

Да, достаточно заблокировать дверь

B

Нет: `open_door` неуправляемо

C

Да, если поставить датчик двери

D

Вопрос некорректен

? Разберём случай

Спецификация «дверь не открывается во время работы станка». `open\_door` — действие оператора. Управляема ли спецификация?

A Да, достаточно заблокировать дверь

B Нет: open_door неуправляемо

C Да, если поставить датчик двери

D Вопрос некорректен

Ответ. Без замка — нет: открытие двери оператором неуправляемо, и спецификация нереализуема. С электромеханическим замком событие становится управляемым, и спецификация реализуема.

“

Синтез не заменяет инженера — он переносит его работу с перебора комбинаций на формулировку смысла.

Ответственность за спецификацию остаётся полностью на человеке

Выводы

- 1 Управляемы начала действий; неуправляемы завершения, отказы и внешние воздействия
- 2 Спецификация реализуема точно тогда, когда она управляема
- 3 Для любой спецификации существует единственный супремальный управляемый подязык
- 4 Алгоритм итеративного удаления состояний сходится за конечное число шагов
- 5 Синтез распространяет ограничения назад по цепочкам неуправляемых событий
- 6 Модульный синтез требует проверки неконфликтности; децентрализованный — соуправляемости
- 7 При частичной наблюдаемости супремум не существует; практическая замена — нормальность

Вопросы для самопроверки

1. Почему событие «операция завершена» неуправляемо? Приведите ошибку от противоположного допущения
2. Сформулируйте условие управляемости и объясните его физический смысл
3. Докажите необходимость управляемости в теореме Рамаджа–Вонэма
4. Почему тривиальный супервизор, запрещающий всё, формально корректен? Что его исключает?
5. Докажите замкнутость класса управляемых подъязыков относительно объединения
6. Зачем в алгоритме синтеза внешний цикл, а не однократное удаление?
7. Что означает обратное распространение по неуправляемым событиям?
8. Когда модульный синтез даёт блокирующий результат и как это обнаружить?
9. Почему при частичной наблюдаемости нет супремального наблюдаемого подъязыка?
10. Какое условие на состав датчиков делает нормальность приемлемой заменой?